

Respuestas a las preguntas de oponencia

El SAT y el vacío de la intersección de una RCG con un lenguaje regular finito

Luis Alejandro Arteaga Morales

Facultad de Matemática y Computación
Universidad de La Habana

2026

- “Su trabajo demuestra que las gramáticas de concatenación de rango permiten modelar la consistencia de *cualquier* fórmula booleana con aridad uno, gracias a la no linealidad, pero reconoce que con ello se pierde la garantía de que el vacío de la intersección sea decidible en tiempo polinomial, garantía que sí existía en SATCF y SATSRC. Siendo así, ¿cuál es el aporte concreto de su tesis respecto a esos antecedentes? Dicho de otro modo: ¿qué problema queda mejor resuelto, o mejor planteado, después de su trabajo que antes de él?”

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridad.

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridad.
- Con RCG, la no linealidad modela **cualquier** fórmula manteniendo la aridad en uno.

Oponencia — pregunta 1 (respuesta)

Punto de partida

Buscar más fórmulas de SAT que se resuelvan en tiempo polinomial, o resolverlas todas, extendiendo SATCF (CFG) y SATSRC (sRCG) a las RCG.

- Con sRCG, el vacío crecía exponencialmente con los cruces: subía la aridez.
- Con RCG, la no linealidad modela **cualquier** fórmula manteniendo la aridez en uno.

El precio

Al salir de las sRCG, el vacío de la intersección es NP-completo en nuestro caso.

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

El aporte concreto

Construimos la **gramática producto** G' : un objeto matemático sobre el que estudiar la intersección (su tamaño, qué predicados son productivos, el costo de decidir el vacío, otras propiedades).

No cierra el camino

Pueden existir subclases no lineales de RCG cuyo vacío de la intersección con un regular finito sea polinomial, aún no descubiertas.

El aporte concreto

Construimos la **gramática producto** G' : un objeto matemático sobre el que estudiar la intersección (su tamaño, qué predicados son productivos, el costo de decidir el vacío, otras propiedades).

Por qué es un aporte

La literatura construía la intersección solo hasta sRCG y no manejaba la no linealidad. Este trabajo sí.

- “En la sección 5.3.2 usted exige que la gramática de entrada sea *no borrante*, es decir, que toda variable del lado izquierdo de una cláusula aparezca también en el lado derecho, y resuelve el caso general apelando a que toda RCG admite una equivalente no borrante. Sin embargo, el motivo de fondo es que una variable borrada (presente solo en el lado izquierdo) ocupa una subcadena que *a la gramática le es indiferente, pero que el autómata sí debe leer*. ¿Por qué optó por transformar la gramática a una equivalente no borrante en lugar de tratar el borrado directamente en la construcción? En particular, ¿consideró precomputar un predicado auxiliar que, para cualquier par de estados (q, q') , derive a la cadena vacía si y solo si ese rango es realizable en el autómata, y añadirlo al lado derecho para fijar el rango de la variable borrada? ¿Qué ventajas y costos tiene cada vía?”

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

Dos vías

- **(A) Transformar** G a una equivalente no borrante **(la elegida)**.

El problema

Una variable borrada le es indiferente a G , pero A sí debe leer su subcadena: hay que fijar su rango a uno realizable.

Dos vías

- **(A) Transformar** G a una equivalente no borrante **(la elegida)**.
- **(B) Predicado auxiliar**: dada la cláusula borrante $A(XY) \rightarrow B(X)$, reconocer las subcadenas que realizan (q, q') (un subautómata) y engancharlas.

$$A[(q_1, q_2)](XY) \rightarrow B[(q_1, q_3)](X) \text{ Reach}[(q_3, q_2)](Y)$$

Por qué elegimos (A)

- Se apoya en un resultado de Boullier: su correctitud ya está demostrada.
- G_{cons} ya es no borrante.

Por qué elegimos (A)

- Se apoya en un resultado de Boullier: su correctitud ya está demostrada.
- G_{cons} ya es no borrante.

(B) también es válida

- Añade los Reach durante el producto; mismo lenguaje.
- Pero exige rehacer la demostración (sin no borrante).

- “La construcción de la gramática producto exige que la gramática de entrada sea no borrante y no combinatoria, y que todas las ocurrencias de una misma variable en una cláusula reciban el mismo rango. Usted afirma que estas restricciones no descartan ningún lenguaje, pues toda RCG admite una equivalente que las cumple. ¿Podría justificar esa afirmación y precisar el costo de transformar una RCG arbitraria en una equivalente no borrante y no combinatoria? ¿Cómo garantiza, además, que la condición de igualdad de rangos para variables repetidas es suficiente para no reconocer cadenas fuera de la intersección?”

No descarta ningún lenguaje

Boullier (1998), encadenando tres equivalencias:

- toda RCG tiene una equivalente no combinatoria;
- de ahí, una equivalente no borrante;
- se conserva la no linealidad, que es lo que da poder sobre las sRCG.

Lemma 8.7.

1. *For any RCG, there is an equivalent non-combinatorial RCG.*
2. *For any non-combinatorial bottom-up erasing RCG, there is an equivalent non-combinatorial bottom-up non-erasing RCG.*
3. *For any non-combinatorial bottom-up non-erasing top-down erasing RCG, there is an equivalent non-combinatorial non-erasing RCG.*

In other words, the possibilities of combinatorial clauses and erasing clauses do not increase the generative capacity of the grammar. For every RCG, there is an equivalent non-combinatorial non-erasing RCG. The crucial property for RCG's being more powerful than simple RCG is the possible non-linearity of the clauses.

Kallmeyer (2010), *Parsing Beyond Context-Free Grammars*, Lemma 8.7, p. 163.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria → tres cláusulas.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria $\rightarrow$ tres cláusulas.
- No borrante (lado derecho, bottom-up): el predicado izquierdo gana un argumento con esa variable.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

¿Cuánto cuesta la transformación?

Boullier la da en tres pasos locales (Propiedades 1–3):

- No combinatoria: cada cláusula combinatoria $\rightarrow$ tres cláusulas.
- No borrante (lado derecho, bottom-up): el predicado izquierdo gana un argumento con esa variable.
- No borrante (lado izquierdo, top-down): se añade un predicado *Any* que acepta cualquier cadena.

Boullier (1998), *A Proposal for a Natural Language Processing Syntactic Backbone*, Properties 1–3, pp. 11–12.

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Se reemplaza por tres cláusulas con predicados nuevos c, c' :

$$\begin{aligned} A(W, X, Y) &\rightarrow c(W, X, Y, W) \\ c(W, X, Y, U) &\rightarrow c'(X, Y, Z) B(W, Z) \\ c'(X, Y, XY) &\rightarrow \varepsilon \end{aligned}$$

Ejemplo 1: no combinatoria

Cláusula combinatoria (XY no es una variable suelta):

$$A(X, Y) \rightarrow B(XY)$$

Se reemplaza por tres cláusulas con predicados nuevos c, c' :

$$\begin{aligned} A(W, X, Y) &\rightarrow c(W, X, Y, W) \\ c(W, X, Y, U) &\rightarrow c'(X, Y, Z) \quad B(W, Z) \\ c'(X, Y, XY) &\rightarrow \varepsilon \end{aligned}$$

$c'(X, Y, XY)$ usa la no linealidad para pegar las piezas.

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Se le da a A un argumento nuevo con Y :

$$A(Y, X) \rightarrow B(X, Y)$$

Ejemplo 2: borrado (lado derecho, bottom-up)

Y aparece solo a la derecha:

$$A(X) \rightarrow B(X, Y)$$

Se le da a A un argumento nuevo con Y :

$$A(Y, X) \rightarrow B(X, Y)$$

Ahora Y está en ambos lados.

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Any acepta cualquier cadena: Y se consume sin restringir nada, mismo lenguaje.

Oponencia — pregunta 3 (respuesta)

Ejemplo 3: borrado (lado izquierdo, top-down)

Cláusula borrante (la Y solo está a la izquierda):

$$A(XY) \rightarrow B(X)$$

Se reescribe añadiendo $\text{Any}(Y)$:

$$A(XY) \rightarrow B(X) \text{Any}(Y)$$

$$\text{Any}(aX) \rightarrow \text{Any}(X), \quad \text{Any}(\varepsilon) \rightarrow \varepsilon$$

(a es un terminal cualquiera)

Any acepta cualquier cadena: Y se consume sin restringir nada, mismo lenguaje.

Es el mismo Any que, al intersectar con A , se vuelve Reach (pregunta 2).

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridad, no el número de cláusulas.

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridad, no el número de cláusulas.
- Propiedad 3: añade $\text{Any}(Y)$ y un bloque fijo de $O(|T|)$ cláusulas.

Por qué es polinomial

- Propiedad 1: cada cláusula combinatoria da a lo sumo 3 (factor constante).
- Propiedad 2: sube la aridad, no el número de cláusulas.
- Propiedad 3: añade $\text{Any}(Y)$ y un bloque fijo de $O(|T|)$ cláusulas.
- G_{cons} ya es no borrante y no combinatoria.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.
- La condición obliga a que todas sus ocurrencias lleven ese par.

Por qué la igualdad de rangos basta

- Una variable repetida es la misma pieza de w en varios predicados.
- En el recorrido real, esa pieza se atraviesa una vez: un solo par de estados.
- **La condición obliga a que todas sus ocurrencias lleven ese par.**
- Sin ella, los índices describirían un recorrido falso, y G' aceptaría cadenas fuera de la intersección.